Guess Close, Search Exact:
A Formally Verified, Model-Free Memory and Mutation Engine for Language-Model Agents

[Author name]
Independent researcher

Code, data and every run: `github.com/devkancheti4-design/neo4d`

October 2026

Abstract

Language models are good at proposing roughly right answers and poor at being exactly right, cheaply, every time. We present *neo4d*, a model-free engine that sits beside a model and supplies what the model lacks: it mutates code token by token and lets the tests judge every variant, it searches the neighbourhood of a model's attempt, and it remembers verified work as placeholder plans it can replay without the model. What it trusts is decided by *RELATE v2*, an eight-bit integer law with no branches, checked on all 2^{32} inputs for undefined behaviour (UBSan and CBMC); its nine principles hold on every reachable input, and eight further properties on all 256 eight-bit inputs, including that a pattern marked barren is weakened but never erased and that a proven pattern is never demoted. We measure the engine with local 4B models, with Claude Haiku and Claude Sonnet run as agents through the same tools, and against a Claude assistant. With neo4d, phi4-mini fixes 62.5% of 176 bugs (30.7% alone, 42.0% with three tries) and completes 47.1% of 85 multi-step agent tasks (25.9% alone, 40.0% with three tries) with fewer calls than retrying. Sonnet and Haiku keep their accuracy (85/85, 84/85) while neo4d takes 32% and 40% of the tasks off them; a replayed plan takes 0.59 s against 20–36 s for a model. In a loop that repeats until it answers, neo4d mutating each attempt lifts phi4-mini from 22 to 25 and gemma3:4b from 18 to 24 of 31 real bugs, with 22–35% fewer calls. With Haiku only choosing which branches to explore, neo4d's search fixes 17 instead of 14 hard two-fault bugs with 60% fewer test runs. We report equally plainly what failed: evidence handed to a small model as a paragraph made it worse (16 vs. 20 of 31), and nothing helped on bugs a strong model already solves. neo4d adds power by doing work, not by advising.

1 Introduction

An agent built on a language model spends most of its budget on two things the model does badly: being exactly right (a one-token slip fails the tests) and doing the same kind of work again (a rename across files costs as many calls the tenth time as the first). Both are things a model-free system can do well, if it has something to tell right from wrong. Tests are that something.

neo4d is built on this division of labour. The model *guesses*: it reads, proposes a direction, writes. neo4d *checks and remembers*: it expands a guess into every single-token variant its mutation operators know, runs the tests on each, and keeps verified work as plans with the task's names abstracted into slots. An integer law, judged exhaustively and verified formally, decides which plans and patterns are trusted, which are only primed, and which have proved barren.

Contributions.

1. **A verified memory law.** RELATE v2 maps a level and six bits of evidence to one of four trust levels with 48 branch-free instructions. We give its principles, the exhaustive and formal checks, the properties that govern barren patterns, and what is *not* proved (§3). Under a noisy teacher it keeps 58.4% precision where its predecessor, without the barren rule, collapses to 1.9%.

2. **Guess close, search exact.** A model's attempt need only land within one token of a fix: neo4d's neighbourhood search and the tests do the rest. Across two local models this beats test-feedback loops with fewer calls (§5.3).
3. **Plans that bypass the model.** Verified tool traces become placeholder macros, matched by the task's phrasing with names removed, replayed and verified in 0.59 s. They double a small model on agent work and take a third to two fifths of the work off Haiku and Sonnet (§5.2).
4. **A model as a filter.** Inverting the roles—neo4d generates and tests, a model only picks branches—improves a search and cuts its cost (§5.4).
5. **Negative results**, reported with the same care (§6), and every run released.

2 The engine

Mutation. Each code line is tokenised and changed one token at a time by the standard operators of mutation testing [1, 2]: relational ($< \leftrightarrow \leq \leftrightarrow = \ \ldots$), arithmetic, logical connectors and negation, constants ($0 \leftrightarrow 1$, `True`↔`False`), control flow (`break`↔`continue`, `return x`→`return None`), and, in a harder set, argument, index, deletion and unary insertion. On the programs used here this gives about 3.4 variants per code line.

Grading. Every variant runs against every test case in a forked process with a time limit. For each case neo4d records pass or fail and the value returned (or the exception's type). Three counts follow: *fixed* (failing cases that now pass), *broken* (passing cases that now fail) and *moved* (failing cases whose output changed at all). *Moved* localises a fault, as in mutation-based fault localisation [3]; *fixed*−*broken* ranks near-fixes. A program's own printed output is discarded; there is no partial credit inside a case.

Plans. From accepted work—a sequence of tool actions (`grep`, `replace`, `write`, `test`, …)—neo4d writes the task's names into slots. A task names things in backticks; its *skeleton* replaces each with $\langle i \rangle$. Names become slots at identifier boundaries in code, between non-alphanumerics in paths (`tests/test_`$\langle 1 \rangle$`.py`), and around regular-expression escapes in patterns (`\b`$\langle 2 \rangle$`\(`). A step repeated over every file a search found becomes one looping step; an answer equal to the number, paths or module names of the files found becomes a derived value. A step whose text a plan cannot fill (the code a fix writes) is marked for the model to write. Matching is exact equality of skeletons; every replay runs on a fresh copy of the project and is accepted or discarded whole.

The exchange. Given a model's attempt, neo4d mutates it—the lines the model changed first—and ships the first variant that passes every test; otherwise it reports what the attempt and its best variants pass, and the model goes again (§5.3).

Deep search. neo4d expands a program by every single-token change, keeps $B = 3$ candidates, and expands those again, up to four changes deep, with duplicates skipped. Kept candidates are chosen either by tests passed (*blind*) or by a model shown the 12 most promising, at most two per line (*filter*). The cost is about $M + dBM$ test runs for M single-token variants and depth d—linear in depth, not M^d.

3 RELATE v2: a verified law for what to trust

Each pattern—a relation, a plan—carries a level: 0 unknown, 1 primed, 2 active (used), 3 proven. At every lesson the body computes six bits of evidence and the law returns the new level:

$$x \ = \ L \mid \textsc{fire} \ll 2 \mid \textsc{newnet} \ll 3 \mid \textsc{pred} \ll 4 \mid \textsc{cheap} \ll 5 \mid \textsc{costly} \ll 6 \mid \textsc{barren} \ll 7.$$

FIRE: the lesson exhibits the pattern; NEWNET: in a place (bug, file, module) it was not seen in; PRED: the pattern, used, was right; CHEAP/COSTLY: what using it costs (for a plan, model calls; in the mutation domain, candidates per line and how many are wrong); BARREN: it has fired at least k times ($k = 10$ here, 100 for bugs) with under 5% success. The law was authored outside this system from a written specification

of nine principles (R1 a costly pattern is never used; R2 silence keeps; R3 first sight primes; R4 a cheap pattern is used at first sight unless R9; R5 a second place generalises; R6 a correct prediction potentiates; R7 proven only by prediction; R8 nothing else raises; R9 a barren pattern weakens unless proven). It compiles to 48 instructions with no conditional branches and no memory operations.

What is verified.

- All 4,294,967,296 `int32` inputs under UBSan: no undefined behaviour, every output in 0..3, and the output depends only on the low eight bits. CBMC [6] proves the same, with no signed overflow.
- The nine principles hold on all 114 reachable words, with 0 violations; no single-cell change to the table escapes them.
- On all 256 eight-bit inputs, the properties of Table 1 hold.

property (all 256 inputs)	holds
A barren, unpredicted, unproven pattern is never active (level ≤ 1)	✓
Barrenness weakens but never erases: never below primed, at most one step down	✓
A proven pattern is never demoted by barrenness (only COSTLY caps it)	✓
A new place cannot re-activate a barren pattern; only a correct prediction can	✓
Without BARREN or COSTLY a level never decreases (monotone)	✓
A correct prediction always makes a pattern active, unless costly	✓
A costly pattern is never active	✓
With the evidence held fixed, every trajectory reaches a fixed point (no oscillation)	✓

Table 1: Properties of RELATE v2, checked by enumeration of its whole input space.

What is not proved. The BARREN bit is a statistic computed by the body; the law guarantees what happens once a pattern is marked (it is weakened, not erased, and can return only by predicting correctly), not that the marking is right. Convergence of the whole system depends on the stream of evidence, not on the law alone.

Why it matters. With a teacher that gives wrong lessons, the barren rule is what keeps memory clean. On 10,000 lessons of the mutation domain, evaluated on held-out modules over three seeds, v1 and v2 cover the same share of real mutants, but their precision differs (Table 2).

wrong lessons	v1: coverage / precision	**v2**: coverage / precision
0%	96.4% / 81.6%	96.3% / **82.0%**
10%	96.3% / 1.9%	96.3% / **58.4%**
30%	96.1% / 1.1%	95.9% / **27.3%**
50%	95.5% / 0.9%	95.4% / **20.0%**

Table 2: A noisy teacher. v1 lets a cheap pattern survive barrenness; v2 does not.

4 Experimental setup

Models. phi4-mini, gemma3:4b and qwen3.5:4b run locally through ollama on one Apple-silicon machine, temperature 0 for a first try and 0.8 for later ones. Claude Haiku and Claude Sonnet run as agents (subagents) through exactly the tools the local models use, one action per command. A Claude assistant (Opus-class, the system used to build and run this study) answered a sample under a handicap (two rounds of actions instead of up to ten).

Benchmarks. All built on the 31 QuixBugs [5] programs whose tests are JSON cases.

- **Bug stream** (176 tasks): each correct program broken on one line by the mutation operators, kept only if its own tests catch it, in one seeded order.
- **Agent work** (85 tasks): the programs as a project (`qb/`, one test file each under `tests/`); six kinds of task—rename a function across files, rename a parameter, add a keyword parameter, count modules using a keyword in code (docstrings excluded), list the modules calling a built-in, fix a bug—each phrased two ways, each accepted only by the tests plus a check of the result itself. Tools: `ls`, `read`, `grep`, `replace`, `write`, `test`, `done`; at most ten actions.
- **Real bugs** (31): the human-written QuixBugs bugs. neo4d's mutations alone fix 5; 26 remain.
- **Hard bugs** (29): each correct program obfuscated (hash-named variables, no docstrings), its function renamed to an opaque name, and broken by two faults on two lines, each fault alone failing the tests, no single change fixing the pair. 18 are reachable by neo4d's operators; 11 contain a fault outside them.

5 Results

5.1 Bug stream: memory that compounds

phi4-mini, 176 tasks	fixed	original line exactly	model calls
model alone	30.7%	29.0%	176
model + tests (3 tries)	42.0%	38.1%	407
neo4d + model	**62.5%**	58.5%	**251**

Table 3: neo4d's memory proposes fixes first; a single passing proposal ships with no model call.

Fixes from memory rose from 38.6% of the first half of the stream to 45.5% of the second; 71 of 74 restored the original line exactly. A perfect writer scores 166/176 through the same harness (its line-placement heuristic loses `return x→return None` bugs). On the first 30 tasks the assistant fixed 28—the harness ceiling—with one answer each, against 7 for phi4-mini alone and 15 with neo4d; neo4d + the assistant also fixed 28, with 19 calls instead of 30.

5.2 Agent work: plans that bypass the model

85 tasks	accepted	second half	model calls	tasks needing the model
phi4-mini alone	25.9%	30.2%	579	85
phi4-mini + 3 tries	40.0%	44.2%	1,406	85
neo4d + phi4-mini	**47.1%**	**55.8%**	**1,225**	–
Claude Sonnet alone	85/85	43/43	418	85
neo4d + Sonnet	**85/85**	43/43	**285**	**58**
Claude Haiku alone	84/85	43/43	401	85
neo4d + Haiku	**84/85**	43/43	**226**	**51**

Table 4: For the large models, model calls are their actions; "neo4d + model" replays trusted plans that need no model and uses the model's own recorded work for every other task (a model starts each task fresh, so its work does not depend on neo4d). Plans of equal strength are tried in a fixed order, so these numbers reproduce exactly.

neo4d's plans completed 3 tasks alone in the first half of the stream and 13 in the second. In the second half neo4d took 19 of 43 tasks off Sonnet and 22 of 43 off Haiku. A replayed plan takes a median 0.59 s, of which 0.46 s is the full test run that verifies it; phi4-mini needs about 20 s per task and the subagents 27–36 s (wall clock, including tool overhead)—35 to 60 times slower. On a 28-task sample the assistant completed 28 (two rounds, one try), phi4-mini 9 alone and 16 with neo4d; neo4d + the assistant completed 28 with the assistant needed on 19.

By kind, neo4d's plans did most for renaming a parameter (9 tasks with no model call) and nothing for renaming a function: phi4-mini completed one rename in fifteen, and the plan it left did not carry to the others.

5.3 Round after round until it answers

On the 31 real bugs, two loops of up to ten rounds that differ only in neo4d: *test feedback* (the model is told what its last program passes and the first failing case) and *model ⇄ neo4d* (the same, and neo4d mutates the model's program, ships any passing variant, and reports its best variants). Neither is given evidence about the original bug up front.

up to 10 rounds, 31 real bugs	test feedback	calls	**model ⇄ neo4d**	calls
phi4-mini	22	154	**25**	**120**
gemma3:4b	18	151	**24**	**98**
qwen3.5:4b	*pending*	*pending*	*pending*	*pending*

Table 5: The model gives a direction, even when wrong; neo4d expands it into hundreds of tested variants.

fixed by round	1	2	3	5	10
per-round counts pending					

Table 6: How fixes accumulate.

5.4 A model as a filter for neo4d's search

29 hard bugs	fixed	reachable (18)	unreachable (11)	model calls	test runs
blind (keeps the 3 passing most tests)	14	13	1	0	5,815
Haiku as the filter	**17**	**16**	1	37	**2,316**
Haiku writing the code, for reference	29	18	11	29	–

Table 7: neo4d generates and tests every program; Haiku only picks which branches go deeper.

The filter fixed every bug the blind search fixed and three more. Writing code directly, Haiku fixed all 29: these are small classical algorithms it recognises even obfuscated; the filter's ceiling is neo4d's operators.

5.5 Clarity for a small model's attention

The first design gave phi4-mini neo4d's evidence as a paragraph before the bug. We also show the same evidence as a single comment on the one or two lines neo4d's mutations point to.

6 What did not work

- **Advice as a paragraph.** With neo4d's evidence before the bug, phi4-mini fixed 16 of 31 with three tries, against 20 without it; an exchange of three rounds with that evidence reached 18. A small model's attention is diluted by text it must weigh.
- **Helping a model that already knows.** Haiku fixed all 31 real bugs at the first submission with or without neo4d, and all 29 hard bugs—at the first submission without neo4d, while with neo4d's evidence two needed a second.

shown	one try	up to 3 tries	calls
pending			

Table 8: The same bug shown plainly, with evidence as a paragraph, and with marks on the suspect lines.

- **Vocabulary for search.** An exploratory test of code search (descriptions to functions over click, rich and pytest) found that learned word-to-code links changed first-time search accuracy by -0.4 to -0.2 points; links to remembered functions helped searching again ($+4.5$ points) and hurt first searches. It was stopped.
- **Nothing from nothing.** A plan needs a success that generalises: phi4-mini renamed a function across files once in fifteen tasks, and the plan did not carry to the others. Eleven of the hard bugs lie outside neo4d's operators and no search of them can fix them.

7 Threats to validity

The benchmarks are small (31, 85, 176 tasks) and each configuration was run once; differences of three bugs may be noise. QuixBugs is public, and every model may have seen it; the hard set reduces but does not remove recall. Two benchmarks inject bugs with the same operators neo4d uses to repair, which favours neo4d's memory and search; the real-bug and agent results do not have this circularity in the bug itself. The agent tasks recur by design, which is where plans pay. The phi4-mini agent run was made before plan ties were ordered deterministically. The assistant that built the study also answered a sample and belongs to the same family as two evaluated models. A check (the tests and a check of the result) is assumed; without one neo4d cannot tell right from wrong.

8 Related work

Mutation testing [1, 2] supplies the operators and mutation-based fault localisation [3] the *moved* signal. Search-based repair such as GenProg [4] evolves patches against tests; neo4d instead searches the neighbourhood of a model's attempt and remembers what passed. Learning-based repair has moved to large models [7, 8], and self-debugging [9] and Reflexion [10] feed test results or reflections back to the model; our test-feedback loop is that baseline. Sampling many programs and filtering by tests [11] spends model calls on breadth; neo4d spends none. Skill libraries [12] and agent-computer interfaces [13] store or shape what agents do; neo4d's plans are verified, slot-abstracted and governed by a formally checked law. The memory law is checked with CBMC [6].

9 Conclusion

A model need not be exactly right if something beside it can search exactly and remember. Under a law whose behaviour is proved over its whole input space, neo4d doubles a small model on agent work, beats test-feedback loops with fewer calls by mutating each attempt, takes a third or more of repeated work off large models at equal accuracy, and makes a model useful as a cheap filter for a search it could not run itself. It does not make a model reason better by advising it, and it cannot create a skill no one has shown it.

Reproducibility. Every number comes from a command in the repository: `neo4d local-model`, `local-agent` (with `--agent-score sonnet,haiku`), `local-hints` (with `--loop`, `--focus`, `--search-score`). Every action of the Claude subagents and every local run is in `results/data/`.

Use of AI. The implementation, experiments and text were produced with Claude Code (Anthropic) under the author's direction; the author chose the questions and the claims. Claude models are also subjects of evaluation here.

References

[1] R. A. DeMillo, R. J. Lipton, F. G. Sayward. Hints on test data selection: help for the practicing programmer. *IEEE Computer* 11(4), 1978.

[2] Y. Jia, M. Harman. An analysis and survey of the development of mutation testing. *IEEE Transactions on Software Engineering* 37(5), 2011.

[3] M. Papadakis, Y. Le Traon. Metallaxis-FL: mutation-based fault localization. *Software Testing, Verification and Reliability* 25(5–7), 2015.

[4] C. Le Goues, T. Nguyen, S. Forrest, W. Weimer. GenProg: a generic method for automatic software repair. *IEEE Transactions on Software Engineering* 38(1), 2012.

[5] D. Lin, J. Koppel, A. Chen, A. Solar-Lezama. QuixBugs: a multi-lingual program repair benchmark set based on the Quixey Challenge. *SPLASH Companion*, 2017.

[6] E. Clarke, D. Kroening, F. Lerda. A tool for checking ANSI-C programs. *TACAS*, 2004.

[7] C. S. Xia, Y. Wei, L. Zhang. Automated program repair in the era of large pre-trained language models. *ICSE*, 2023.

[8] Z. Fan, X. Gao, M. Mirchev, A. Roychoudhury, S. H. Tan. Automated repair of programs from large language models. *ICSE*, 2023.

[9] X. Chen, M. Lin, N. Schärli, D. Zhou. Teaching large language models to self-debug. *ICLR*, 2024.

[10] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, S. Yao. Reflexion: language agents with verbal reinforcement learning. *NeurIPS*, 2023.

[11] Y. Li et al. Competition-level code generation with AlphaCode. *Science* 378, 2022.

[12] G. Wang et al. Voyager: an open-ended embodied agent with large language models. arXiv:2305.16291, 2023.

[13] J. Yang et al. SWE-agent: agent-computer interfaces enable automated software engineering. *NeurIPS*, 2024.